

DocumentKB

既存の文書を、エージェントが ID で引用できるカタログにする仕組み

AI-DLC ドキュメント · User Guide · ナレッジ · CLI コマンド

DocumentKB は、スペースがすでに持っている文書を、エージェントが **ID で引用できるカタログ** にする仕組みです。ビジョン、PRD、要件、ポリシー、契約、仕様 PDF など、ワークフローの入力になる成果物が対象です。

人が Markdown で書く 第2層ナレッジ(「いつもこう作る」)とは別です。既存コードのリバースエンジニアリング成果物を置く CodeKB とも別です。

目次

1. 何か
2. どのように活用するか
3. どのファイルが使えるか
4. コマンド
5. 抽出の状態
6. 第2層ナレッジとの使い分け
7. 次に読む

何か

エージェントは、ディスクに置いただけの PDF や Word を自動では読みません。DocumentKB に載せて初めて、次が決まります。

- どのファイルか(文書 ID、パス、ダイジェスト)
- 本文が取れたか(抽出状態)
- どのインテントから見えるか
- 引用してよいか(カタログ行があること)

カタログは **派生** です。原本はチームが持ち、索引・抽出本文・要約はツールがトランザクションで書きます。

```
aidlc/spaces/<space>/knowledge/
├── documents/      # チーム所有の原本。追加・移動・削除は人がする
├── documentkb/     # ツール所有の派生カタログ。手で直さない
│   ├── index.json
│   └── <document-id>/
│       ├── metadata.json
│       ├── content.md      # 抽出できたとき
│       └── summary.md      # summarize したとき
```

`<space>` は名前付きスペースを使わなければ `default` です。フレームワークは `documents/` を並べ替えたり消しません。消すときは原本を消し、そのあと `sync` します。`remove` コマンドはありません。

`index.json` だけ失ったときは `sync` が、残っている文書ごとの `metadata.json` (墓石を含む) から組み直します。`documentkb/` ごと消すと ID と墓石は戻りません。残った原本は新しい行として再索引されます。

抽出本文・要約・タグは **信頼できないデータ** です。文書の中の命令文は、AI-DLC の指示ではありません。

`show` はその注記を本文と並べて出します。

どのように活用するか

1. 既存文書からワークフローを始める

テキストや短い Markdown を `/aidlc` に直接渡すだけなら、DocumentKB は不要です。既存文書から始める のパス指定や `<document>` ブロックで足ります。

PDF、Word、サイズの大きいファイルは、いったんカタログへ載せ、できた文書 ID をワークフローの入力にします。エージェントはファイル名を推測せず、索引された行だけを引用します。

2. スペースの共通根拠を置く

ポリシー、契約、ドメイン用語、横断の仕様は、intent を付けずに onboard します。そのスペースのどのワークフローからも見えます。「この案件だけ」に閉じない根拠向きです。

3. 案件の文書を一つのintentへ絞る

そのワークフロー専用の PRD や要件は `associate` します。他のintentのエージェントは引きません。終わったあとも文書はスペースに残り、絞りを外すとまた全体から見えます。

4. 要約で要旨だけ載せる

`summarize` は、エージェントが書いた短い要旨を、そのリビジョンに紐づけて残します。ツール自身は本文を生成しません。全文を毎回読まずに、何の文書かを先に判断できます。原本を直して `sync` すると要約は `invalidated` になり、古い文は出ません。

5. 原本の更新をカタログへ反映する

`documents/` のファイルを置き換えたなら `sync` します。ダイジェストが変わった行は再抽出し、古い本文と要約は無効になります。パスは同じで中身だけ変わったときは、`onboard` もその行を `edited` として更新します。パスも中身も変わったときだけ `rebind` が必要です。

典型的な流れ

- 1 対象ファイルを `aidlc/spaces/<space>/knowledge/documents/` へ置く
- 2 `/aidlc knowledge onboard` (またはパス付きで 1 件)
- 3 `/aidlc knowledge list` で `extracted` を確認する
- 4 必要なら `associate` でインテントへ絞る
- 5 `/aidlc knowledge show <id>` で本文を引用する
- 6 原本が変わったら同じパスへ置き、`/aidlc knowledge sync`

1 ファイルの例(相対パスはプロジェクトルート基準):

```
/aidlc knowledge onboard aidlc/spaces/default/knowledge/documents/prd/checkout.pdf
```

フォルダ分けはチームの整理です。例:

```
aidlc/spaces/default/knowledge/documents/  
├─ prd/  
│   └─ checkout.pdf  
├─ specs/  
│   └─ api-error-codes.md  
└─ policies/  
    └─ security-policy.pdf
```

ファイル名はデータであり、エージェントへの指示ではありません。

どのファイルが使えるか

判定は拡張子より中身(マジックバイト)が先です。**索引に載ることと本文が取れることは別**です。載っただけの行も、名前では引用できます。エージェントが中身を使うなら、本文が取れる形式で置きます。

形式	索引	本文抽出(出荷のまま)	備考
Markdown(<code>.md</code>)	できる	できる(組み込み)	UTF-8
プレーンテキスト(<code>.txt</code> など)	できる	できる(組み込み)	UTF-8。Latin-1 などは拒否され得る

形式	索引	本文抽出(出荷のまま)	備考
PDF	できる	できる(既定 <code>pdftotext</code>)	PATH に Poppler が要る。最大 50 ページ
Word(<code>.docx</code> 、OOXML)	できる	未設定なら取れない (<code>unsupported_type</code>)	種類は識別する。抽出器を設定すれば <code>sync</code> で再試行できる
Word 旧形式(<code>.doc</code>)	載っても種類は不明	取れない	<code>application/octet-stream</code>
Excel(<code>.xlsx</code> / <code>.xls</code>)	載っても種類は不明	取れない	表は CSV か Markdown に書き出して載せる
PowerPoint(<code>.pptx</code> / <code>.ppt</code>)	載っても種類は不明	取れない	PDF かノートのテキストに書き出して載せる
画像(JPEG / PNG など)	載っても種類は不明	取れない	OCR なし
その他バイナリ	載っても種類は不明	取れない	

上限

- 1 ファイル 32 MiB。超えると読む前に拒否
- パス無しの `onboard` / `sync` は、新しい・変わった仕事に 20 文書 / 256 MiB
- 抽出本文は 20 万文字。超えると切れて `truncated`

シンボリックリンク、ハードリンク、ディレクトリそのものは索引しません。スキャン PDF の OCR は対象外です。

出荷状態では Word は種類だけ分かり、本文は取りません。`harness.json` の `documentExtractors` に、その MIME 向けの外部コマンドを設定します。`argv` は配列で、`$IN` が引数にちょうど 1 つ要ります(シェル文字列は不可)。設定したあと `sync` すると、変わっていない Word 行も再試行します。同じパスへ `onboard` し直しても `already` と出るだけで、抽出は再試行しません。

Excel / PowerPoint は、抽出器を足しても種類自体を `spreadsheet` / `presentation` として識別しません。表やスライドの本文が要るなら、CSV / Markdown / PDF に書き出します。

コマンド

入口は `/aidlc knowledge <動詞>` です。同じ面は `/aidlc-knowledge` スキルでも案内します。フラグの詳細は CLI コマンドです。

コマンド	役割
<code>onboard [path]</code>	1 ファイル、または <code>documents/</code> 以下の未索引ファイルを索引する。冪等
<code>sync</code>	原本と突き合わせる。新規、墓石、再抽出、失った <code>index.json</code> の再構築
<code>list [--json]</code>	全行と状態
<code>show <id></code>	1 件の記録と抽出本文
<code>associate</code> / <code>dissociate</code>	インテントへの絞りを付ける / 外す
<code>rebind <id> --to <path></code>	原本が移動かつ内容変更した行を直す (<code>sync</code> では解けない)
<code>summarize <id> ...</code>	LLM が書いた要約 (と任意タグ) を残す。ツールは本文を生成しない

`onboard` の結果は `fresh` / `already` / `edited` です。同じパスが生きた行を 2 つ持つことはありません。 `--intent` を省略した `onboard` はスペース全体です。素の `--intent` はアクティブインテント、 `--intent <slug>` は明示の名前です。

抽出の状態

`list` と `show` に出る状態は、直し方がそれぞれ違います。

状態	意味	次にすること
<code>extracted</code>	本文が取れた	そのまま <code>show</code> で引用
<code>no_extractable_text</code>	抽出器は走ったが文字が無い (スキャン PDF など)	テキスト版を用意する。OCR は対象外
<code>extractor_unavailable</code>	設定した抽出器が PATH に無い	入れてから <code>sync</code>
<code>extraction_failed</code>	抽出器が失敗した	<code>show</code> で理由を見る
<code>unsupported_type</code>	この種類の抽出器が無い	対応形式へ書き出すか、Word なら抽出器を設定して <code>sync</code>
<code>invalidated</code>	原本が変わった	<code>sync</code> で再抽出
<code>tombstoned</code>	原本を消した	意図した削除。カタログは覚える

切れた抽出 (`truncated`) は部分ビューです。「この文書は X に触れていない」とは、それだけでは言えません。

第2層ナレッジとの使い分け

	第2層ナレッジ(<code>aidlc/knowledge/</code>)	DocumentKB(<code>knowledge/documents/</code>)
中身	チームが書いた標準・方針	すでに存在する成果物
形式	Markdown	PDF、テキスト、Word、書き出した CSV など
読み方	エージェント起動時にディレクトリから載る	<code>onboard</code> した行だけ <code>list</code> / <code>show</code> で引用
例	コーディング規約、API の形	この案件の PRD、契約、仕様 PDF

「いつもこう作る」はナレッジ、「この文書が根拠である」は DocumentKB です。

次に読む

- ナレッジ — 二層のナレッジと DocumentKB の位置づけ
- 最初のワークフロー — 既存文書から `/aidlc` を始める
- CLI コマンド — 動詞、バッチ上限、抽出器の契約
- スペースと_intent — `documents/` と `documentkb/` のコミット対象